

The app that we are going to build is called the **ToDo app**. With this app, users can list the tasks that need to be done and delete them when completed.

Now let's go to the home component, which is where we will build our app's UI and logic.

The home component has three files:

- *home.component.ts* - In this file, we will add the logic of our app, i.e., the logic to add and delete the task in our app.
- *home.html* - This file consists the UI part of the app; we will be using HTML here.
- *home.scss* - This file contains all the CSS styling of the app and the SASS variables specific to this page.

Creating the UI and the logic of the app

Figure 3 shows the wireframe of the app we are creating. It has a text input where the user can enter the task, a button to add that task to the list, the list to display the added tasks, and a *Delete* button to delete the task when it is completed.

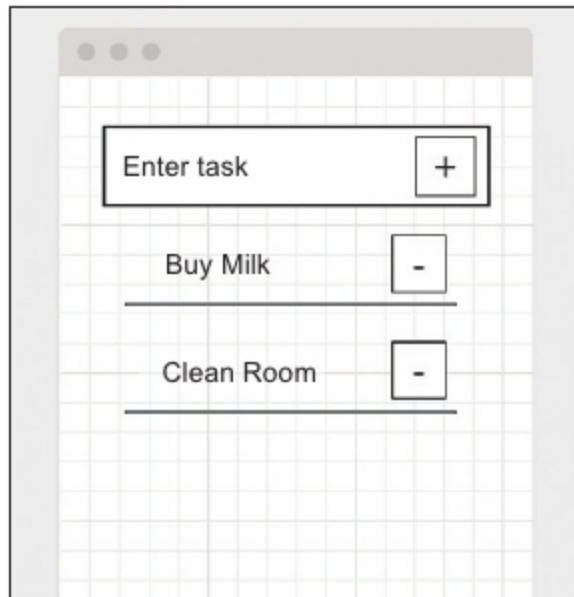


Figure 3: Wireframe diagram of the app

Now let's open the code editor and start coding.

First, let us design the UI of the app, open *home.component.html*, and add the following HTML code:

```
<ion-header>
  <ion-toolbar>
    <ion-title>
```

```
      <div class="item-note" item-end>
        <ion-button color="primary"
          (click)="addTask()">
          <ion-icon name="add"></ion-icon>
        </ion-button>
      </div>
    </ion-item>
  </div>
</ion-list>
</ion-content>

  <div class="item-note"
    slot="end">
    <ion-button color="danger"
      clear (click)="deleteTask(i)">
      <ion-icon name="trash"></
    ion-icon>
  </ion-button>
</div>
</ion-item>
</ion-list>
</div>
</ion-content>
```

We have added `<ion-header>` with `<ion-toolbar>` and `<ion-title>`. This displays the title of the app at the top. Then we have added our UI components inside `<ion-content>`. The `<ion-input>` component is where the user will enter the task, and the `<ion-button>` component when clicked will add the task to `<ion-list>`.

`<ion-list>` will display the list of tasks added to it. As you can see, we are iterating over the `taskList` variable using the `*ngFor` Angular directive. Then, finally, we have added an `<ion-button>` to each item in the list; when this button is clicked, the task will get deleted.

Here, we are calling the `deleteTask(i)` function and we are also passing the current index; this deletes only the list item matching the current index.

Tip: You can read more about Ionic components and how to use them from the official documentation at <https://ionicframework.com/docs/components>.

To add the logic to the app, open *home.component.ts* and add the following code:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
})
export class HomePage {

  taskName: any = ''; // Entered Text
  taskList = []; // Array to store tasks

  constructor() {}

  // addTask Function
  // First we check if the text is
  // entered or not in input box by
  // verifying if length > 0
  // If length is greater than 0, then
  // only we add taskName to taskList array
  // After adding we reset the taskName
  addTask() {
    if (this.taskName.length > 0) {
      let task = this.taskName;
      this.taskList.push(task);
      this.taskName = '';
    }
  }

  // deleteTask Function
  // When user clicks the delete task
  // button, this function is called with
  // index i as parameter
  // Since tasks are added to taskList,
  // we delete the task at index i using
  // splice() array method
```